

SYSTEM DESIGN

PHASE 2 — NETWORKING

🌐 High-End Interview Notes • Simple English • Diagrams • Examples

□ From Browser to Database

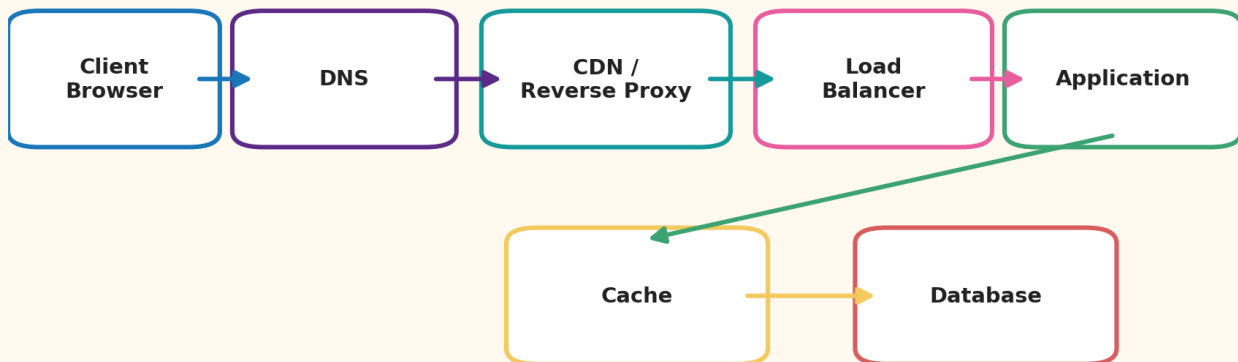


Figure 1 — The complete request journey

🎯 Chapter goal

By the end, you should be able to explain what happens when a user types a URL and presses Enter — from DNS resolution to the application and database — and explain the major networking choices behind that journey.



1. IP Addresses and Ports

An IP address identifies a device or network interface. A port identifies a particular network service or application endpoint on that device.

ELI10

IP address = the building address. Port = the room/door where a particular service is listening.

Concept	Simple meaning	Example
IPv4	32-bit address written as four numbers	192.168.1.10
IPv6	128-bit address with a much larger address space	2001:db8::1
Port	Number identifying a network service	80, 443, 22
Socket	Endpoint used by a network connection	IP + Port



Common ports to remember

Port	Typical use
80	HTTP
443	HTTPS
22	SSH
53	DNS
3306	MySQL
5432	PostgreSQL



Private vs public IP

Private IPs are used inside local networks. Public IPs are reachable through the public Internet. NAT often allows many private devices to share one public IP.



Interview connection

You do not need to memorize every port for System Design. Understand that clients connect to services through network addresses and ports, while firewalls and security groups can control which ports are reachable.

🔍 2. DNS — Domain Name System

DNS converts human-friendly domain names such as `example.com` into IP addresses that computers can use to connect to servers.

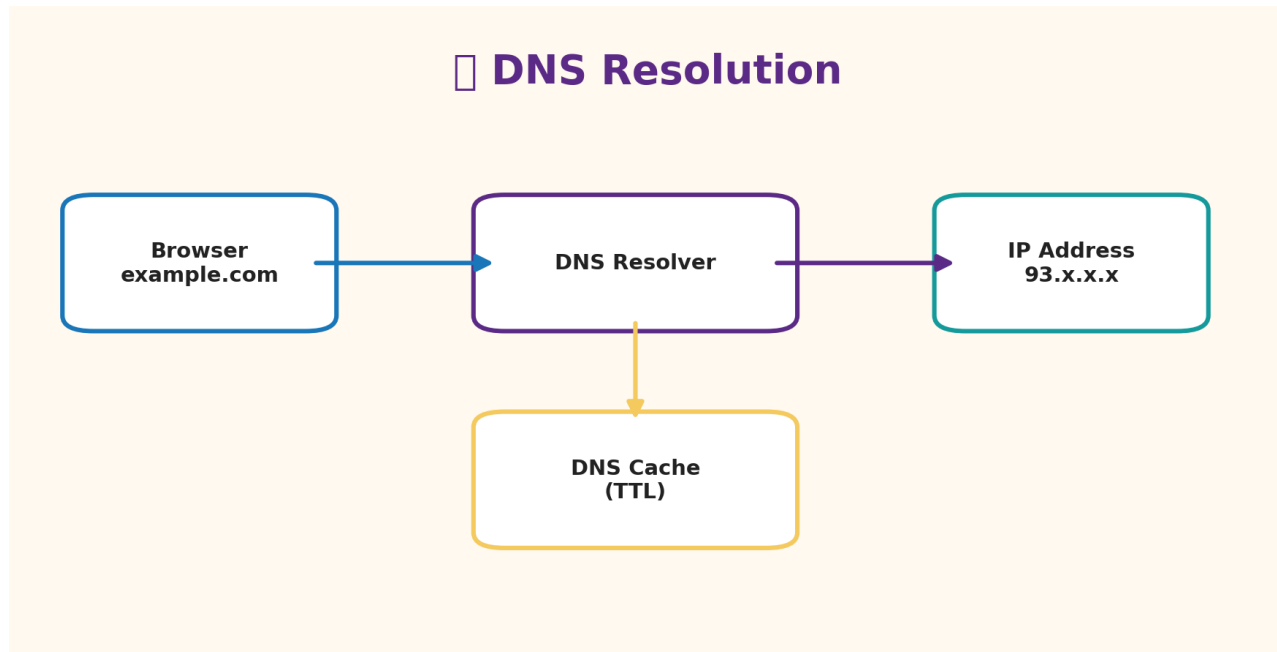


Figure 2 — Simplified DNS resolution

💡 Why DNS exists

Humans remember names more easily than IP addresses. DNS provides a naming system that lets applications locate services without hard-coding an IP address.

⚙️ Simplified DNS journey

1. Browser checks its DNS cache.
2. The operating system or local resolver cache may already know the answer.
3. Recursive DNS resolver looks up the domain if needed.
4. Authoritative DNS servers provide the record.
5. Resolver returns the IP and the result can be cached for its TTL.
6. The client connects to the returned address.

📄 Important DNS records

Record	Purpose	Example idea
A	Maps a name to IPv4	<code>example.com</code> → <code>93.x.x.x</code>
AAAA	Maps a name to IPv6	<code>example.com</code> → IPv6
CNAME	Alias from one name to another	<code>www</code> → <code>example.com</code>
MX	Mail server routing	<code>mail.example.com</code>
TXT	Text/verification/policy data	Domain verification

⚠️ Interview point

DNS caching means the IP returned to a user is not necessarily looked up from authoritative DNS on every request. TTL controls how long a cached answer may be used.



3. TCP vs UDP

TCP and UDP are transport-layer protocols. The choice affects reliability, ordering, connection behavior, and latency.

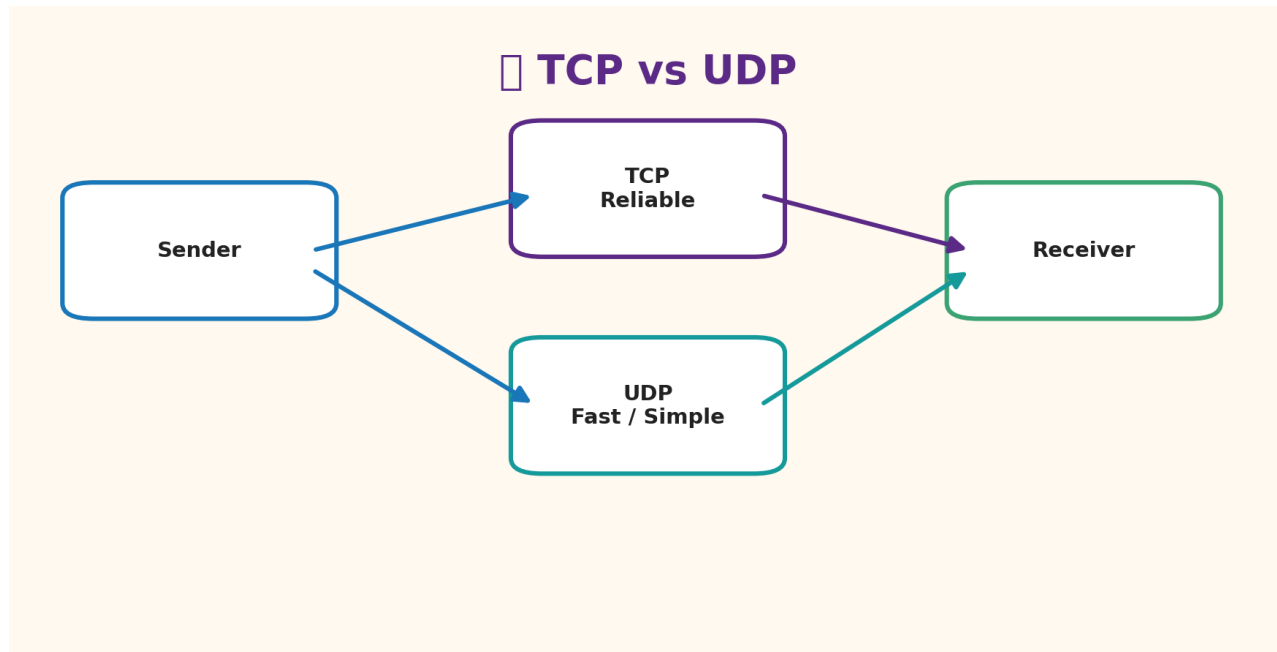


Figure 3 — Simplified TCP and UDP behavior

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Acknowledgements + retransmission	No built-in delivery guarantee
Ordering	Ordered byte stream	No ordering guarantee
Overhead	Higher	Lower
Examples	HTTP/1.1, HTTP/2, SSH	DNS, real-time use cases; QUIC uses UDP



TCP — simple mental model

TCP behaves like a careful courier: establish a connection, send data, confirm delivery, and retransmit missing data.



UDP — simple mental model

UDP behaves like sending packets without waiting for delivery confirmation. This can reduce protocol overhead, but the application/protocol above it must handle reliability if needed.



Interview tip

Do not say “UDP is always faster.” Real performance depends on the application, network conditions, packet loss, congestion, and the protocol built on top.

4. HTTP and HTTPS

HTTP is an application-layer protocol used for communication between clients and servers. HTTPS is HTTP protected by TLS.



Remember

HTTP = communication. HTTPS = HTTP + TLS-protected connection.

HTTPS = HTTP + TLS



Figure 4 — HTTPS protects the HTTP connection with TLS



HTTP request/response

A client sends an HTTP request. The server processes it and returns an HTTP response.

Request/response part	Example
Method	GET
Path	/products/42
Headers	Authorization, Accept, User-Agent
Body	JSON for POST/PUT/PATCH
Response status	200 OK
Response headers	Content-Type, Cache-Control, Set-Cookie
Response body	JSON / HTML / file



Why HTTPS matters

- Confidentiality — protected traffic should not be readable by an attacker on the network.
- Integrity — unauthorized changes to protected traffic should be detectable.
- Authentication — certificates help the client verify the server identity.

5. TLS / SSL Basics

TLS (Transport Layer Security) protects data exchanged over a network. SSL is the older predecessor; modern systems use TLS.

High-level TLS handshake

7. Client and server agree on supported cryptographic parameters.
8. Server presents a certificate identifying the server/domain.
9. The parties establish shared cryptographic keys.
10. Application data is then protected using symmetric encryption.

Certificate basics

A certificate binds a domain identity to a public key and is signed through a trusted certificate-authority chain. Browsers use this trust model to help validate HTTPS servers.

ELI10

Think of TLS as creating a private, tamper-resistant tunnel between your browser and the server. The cryptography is more complex, but this is the right System Design mental model.

TLS termination

TLS may terminate at a CDN, reverse proxy, or load balancer. Depending on the security design, traffic from that layer to the application servers may also use encryption.



6. HTTP Methods and Status Codes



HTTP methods

Method	Typical meaning	Example
GET	Read / retrieve	GET /users/42
POST	Create or trigger an action	POST /orders
PUT	Replace a resource representation	PUT /users/42
PATCH	Partially update a resource	PATCH /users/42
DELETE	Delete a resource	DELETE /users/42



Status code families

Family	Meaning	Examples
2xx	Success	200, 201, 204
3xx	Redirection / cache-related behavior	301, 302, 304
4xx	Client/request problem	400, 401, 403, 404, 409, 429
5xx	Server-side failure	500, 502, 503, 504



Common interview confusion

401 usually means authentication is required or failed. 403 means the server understood the request but refuses authorization. 404 means the requested resource was not found, although some APIs intentionally hide resource existence.



Idempotency

An operation is idempotent when repeating the same operation has the same intended final effect. GET, PUT, and DELETE are generally designed to be idempotent; POST is not inherently idempotent. Payment/order APIs often use idempotency keys to prevent duplicate processing.

7. Headers, Cookies, and Keep-Alive

HTTP headers

Headers carry metadata about a request or response.

Header	Purpose
Content-Type	Describes body format, such as application/json
Authorization	Carries authentication credential/token information
Accept	Describes response formats the client accepts
Cache-Control	Caching instructions
Cookie	Sends cookies stored by the browser
Set-Cookie	Server asks browser to store/update a cookie
User-Agent	Describes client software

Cookies

Cookies are small pieces of data associated with a website. Browsers can automatically send matching cookies with requests.

- Session cookies can help a server identify a logged-in session.
- Secure limits a cookie to HTTPS transmission.
- HttpOnly prevents JavaScript from directly reading the cookie.
- SameSite helps reduce certain cross-site request risks.

Keep-Alive / persistent connections

Opening a new connection has overhead. Persistent connections allow multiple HTTP requests/responses to reuse a connection when supported, reducing repeated connection setup.

Simple idea

Without reuse: connect → request → close → connect again. With reuse: connect once → many requests → close later.

🚀 8. HTTP/1.1 vs HTTP/2 vs HTTP/3

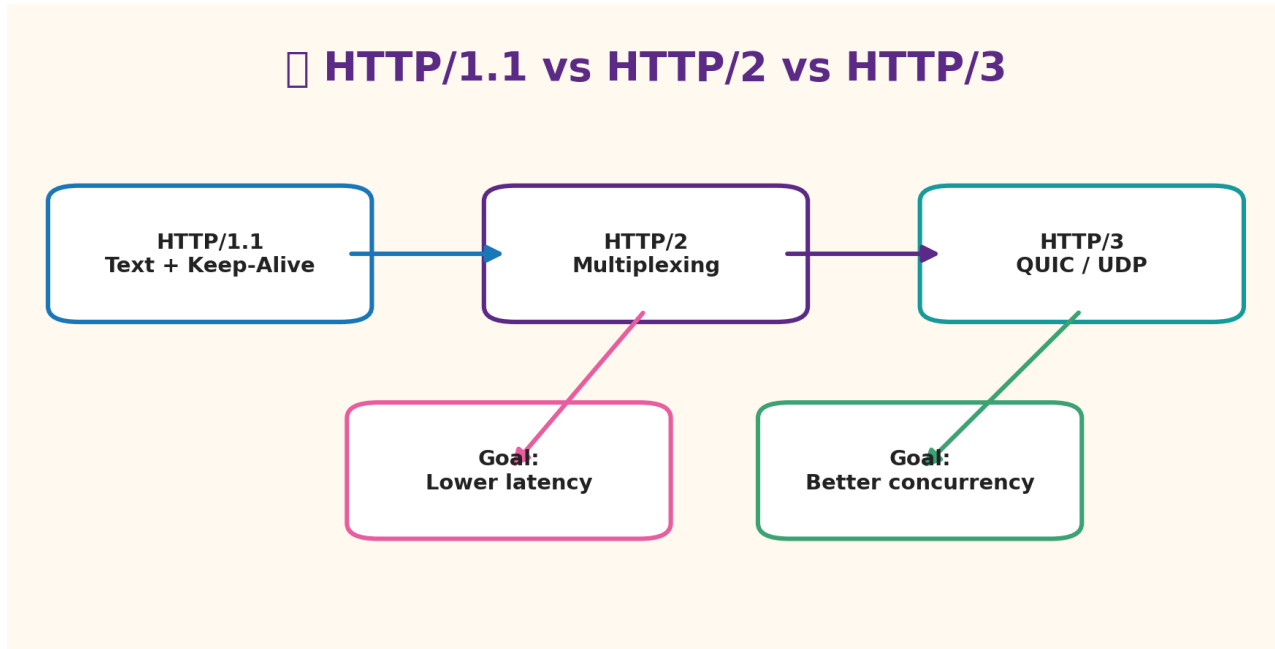


Figure 5 — The evolution of HTTP transport and concurrency

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC over UDP
Multiplexing	Limited; browsers commonly used multiple connections	Multiple streams on one TCP connection	Multiple streams at the QUIC layer
Header compression	No HPACK	HPACK	QPACK
Encryption	Usually HTTPS in modern deployments	Usually HTTPS	TLS integrated into QUIC
Main goal	Simple web communication	Better concurrency and efficiency	Lower connection/setup latency and improved behavior on changing networks

1 HTTP/1.1

HTTP/1.1 introduced persistent connections. Browsers commonly opened multiple connections to improve parallelism.

2 HTTP/2

HTTP/2 adds multiplexing, allowing multiple streams to share one TCP connection, plus compressed headers and other efficiency improvements.

3 HTTP/3

HTTP/3 uses QUIC, which runs over UDP and provides reliable, ordered streams at the QUIC layer. This helps reduce transport-level head-of-line blocking and improves connection behavior on changing networks.

🔗 Interview answer

Remember: HTTP/1.1 → persistent connections; HTTP/2 → multiplexed streams over TCP; HTTP/3 → QUIC

over UDP with modern TLS integration.

9. CDN and Edge Caching

A Content Delivery Network (CDN) is a distributed network of edge locations that can serve cached content closer to users.

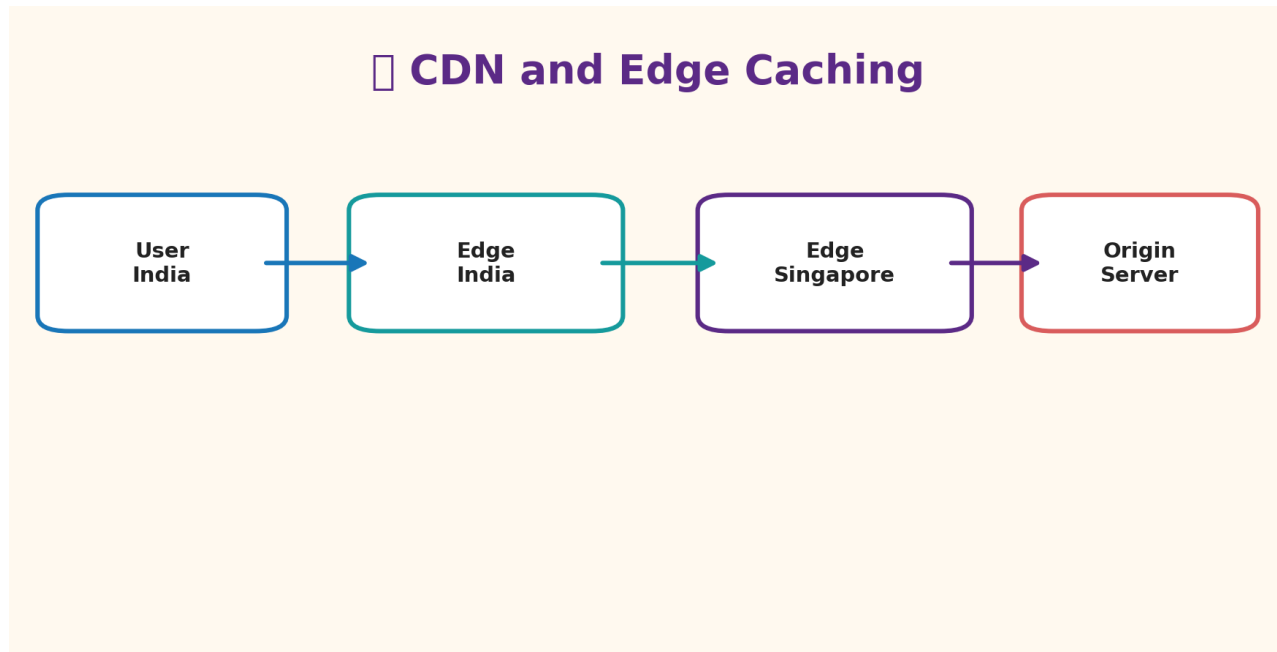


Figure 6 — User traffic can be served from a nearby edge

⚡ Why CDN?

- Reduce latency by serving content closer to users.
- Reduce traffic reaching the origin server.
- Handle large amounts of static-content traffic.
- Absorb some traffic spikes.
- Provide additional security and traffic-management capabilities depending on the provider.

📦 What is commonly cached?

- Images
- CSS and JavaScript
- Fonts
- Video segments
- Public API responses when safe and correctly configured

🔄 Edge caching flow

11. User requests a resource.
12. CDN edge checks its cache.
13. Cache hit → edge returns the object.
14. Cache miss → edge fetches from the origin.
15. Edge stores the object according to caching rules and returns it.

⚠️ Important

Not every response should be cached. Personalized or sensitive responses need careful cache-control

rules so one user's data is never served to another user.

🌐 10. Complete Request Flow

□ From Browser to Database

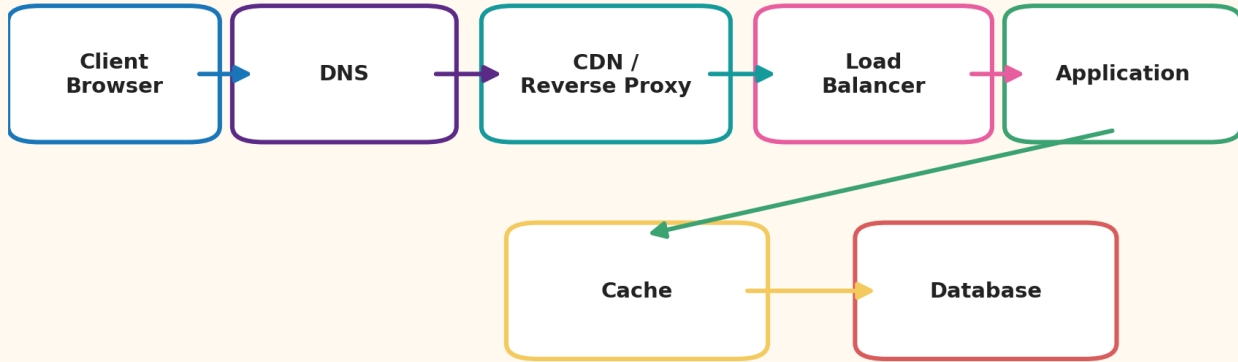


Figure 7 — Client → DNS → CDN/Reverse Proxy → Load Balancer → Application → Cache/Database

🔍 Step-by-step

16. Client: browser/mobile app creates a request.
17. DNS: hostname is resolved to an address/service endpoint. Browser/OS/resolver caches may avoid a fresh lookup.
18. CDN / Reverse Proxy: request may be served at the edge or routed toward the origin. TLS may terminate here.
19. Load Balancer: if multiple application servers exist, traffic is distributed to a healthy backend.
20. Application: backend authenticates, validates, applies business logic, and decides what data is needed.
21. Cache: application may check Redis or another cache first.
22. Database: if data is not cached or must be persisted, application communicates with the database.
23. Response: result travels back through the relevant layers to the client.

🔗 Realistic simplified flow

Browser → DNS → CDN/Reverse Proxy → Load Balancer → API Server → Redis → Database → API Server → Load Balancer → CDN/Proxy → Browser

💧 Big picture

This request flow connects almost everything you will learn later: networking, caching, load balancing, databases, scalability, security, and reliability.

11. Interview Cheat Sheet

Topic	One-line answer
IP + Port	IP identifies the network endpoint; port identifies the service/application endpoint.
DNS	Maps human-friendly names to network addresses using distributed naming and caching.
TCP	Reliable, ordered transport with connection management and retransmission.
UDP	Connectionless transport with lower protocol overhead and no built-in delivery guarantee.
HTTP	Application-layer request/response protocol for web communication.
HTTPS	HTTP protected by TLS.
TLS	Provides confidentiality, integrity, and server authentication using cryptography and certificates.
HTTP methods	GET read, POST create/action, PUT replace, PATCH partial update, DELETE remove.
Status codes	2xx success, 3xx redirect/cache, 4xx client issue, 5xx server issue.
Headers	Metadata about requests/responses.
Cookies	Browser-stored data automatically sent with matching requests.
Keep-Alive	Reuse connections to reduce repeated connection setup.
HTTP/2	Multiplexing and compressed headers over TCP.
HTTP/3	HTTP over QUIC, which runs over UDP.
CDN	Serves cacheable content from distributed edge locations closer to users.

12. Questions You Should Be Able to Answer

- Why do we need DNS?
- What happens after I type a URL and press Enter?
- What is the difference between an IP address and a port?
- Why does HTTPS use TLS?
- Why is TCP reliable while UDP is not?
- When might an application prefer UDP/QUIC-style transport?
- What is the difference between 401 and 403?
- What does idempotent mean?
- Why are HTTP headers needed?
- What is a cookie?
- Why does connection reuse matter?
- What problem does HTTP/2 multiplexing solve?
- Why was HTTP/3 built around QUIC over UDP?
- What does a CDN do?
- What happens on a CDN cache hit vs cache miss?
- Where can TLS terminate?
- Where does a load balancer fit into the request path?

Final memory

When you see a URL, think: NAME → ADDRESS → CONNECTION → SECURE CHANNEL → HTTP REQUEST

→ EDGE/PROXY → LOAD BALANCER → APP → CACHE/DB → RESPONSE.